

CS486 SOLUTION - TEST 5

Based on the FINAL REVIEW structure; total: 10 points

Question 1 - ER Models

[1 pt]

Solution. Conference key ConfAcronym. Track is owner-qualified by Conference with key (ConfAcronym, TrackName). Paper has global key PID and belongs to exactly one Track. Author has key AID. Writes is M:N between Author and Paper with attribute AuthorOrder and key (AID, PID), plus UNIQUE(PID, AuthorOrder) if each position is unique. Review is associative between Reviewer and Paper; a natural key is (ReviewerID, PID) if at most one review per reviewer-paper pair; if RID is used as a surrogate, enforce UNIQUE(ReviewerID, PID).

Question 2 - Query Languages

[2 pt]

Solution. (a)

$$\pi_{w_1.PID}(\sigma_{w_1.PID=w_2.PID \wedge w_1.AID \neq w_2.AID}(\rho_{w_1}(Writes) \times \rho_{w_2}(Writes))).$$

(b) Using extended calculus with AVG:

$$\{a \mid Author(a) \wedge \forall p((Paper(p) \wedge Writes(a, p)) \rightarrow AVG\{r.Score \mid Review(r) \wedge r.PID = p.PID\} \geq 3)\}.$$

A pure classical TRC without aggregates cannot directly express AVG; the question explicitly permits the aggregate extension.

(c)

```
WITH a AS (
  SELECT p.Conf, p.PID, AVG(1.0*r.Score) avg_score, COUNT(*) n
  FROM Paper p JOIN Review r ON r.PID=p.PID
  GROUP BY p.Conf, p.PID
  HAVING COUNT(*) >= 3
), x AS (
  SELECT *, DENSE_RANK() OVER(PARTITION BY Conf ORDER BY avg_score DESC) rk
  FROM a
)
SELECT Conf, PID, avg_score FROM x WHERE rk=1;
```

Question 3 - Database Constraints

[1 pt]

Solution. Use CHECK(Score BETWEEN 1 AND 5) and CHECK(Confidence BETWEEN 1 AND 3). Conflict-of-interest forbidden state:

$$\neg \exists r \exists w (Review(r) \wedge Writes(w) \wedge r.PID = w.PID \wedge r.ReviewerID = w.AID).$$

Implement with an INSERT/UPDATE trigger on Review that rejects rows for which EXISTS (SELECT 1 FROM Writes WHERE Writes.PID=inserted.PID AND Writes.AID=inserted.ReviewerID). Also protect relevant Writes updates if authorship can change after reviews exist.

Question 4 - Serialization & Concurrency**[2 pt]**

Solution. B gives $T_1 \rightarrow T_2$. C gives $T_2 \rightarrow T_3$ and $T_2 \rightarrow T_1$ plus $T_3 \rightarrow T_1$ before $w_1(C)$. A gives $T_1 \rightarrow T_3$ and $T_3 \rightarrow T_2$. Cycles exist, e.g. $T_1 \rightarrow T_2 \rightarrow T_1$, so not conflict-serializable.

Read-from: $r_2(B)$ reads $w_1(B)$, so T_2 depends on T_1 , but commit order c_2, c_1, c_3 has T_2 commit before T_1 ; therefore not recoverable. $r_3(C)$ reads $w_2(C)$, also dirty. Thus not cascadeless.

Strict 2PL holds write/exclusive locks until commit/abort (and in strict implementations shared locks may be released earlier depending on strict vs rigorous 2PL). This prevents other transactions from reading or overwriting uncommitted writes, eliminating dirty reads/writes and cascading aborts from those writes.

Question 5 - Recovery**[1 pt]**

Solution. Winners: T_1 and T_2 . Loser: T_3 . REDO-only means redo committed updates that may not yet be on disk: $A := 100$ for T_1 and $B := 200$ for T_2 . Ignore T_3 's C and D updates because T_3 did not commit. No UNDO is needed because deferred updates from losers were never installed. The completed checkpoint means transactions completed before it need not generally be rescanned from the beginning; recovery can start from/check around the checkpoint and active list according to the protocol.

Question 6 - Functional Dependencies & Normalization**[1 pt]**

Solution. AB is a key by $AB \rightarrow C$ and $B \rightarrow D \rightarrow E \rightarrow F$. CB is also a key because $C \rightarrow A$, then AB gives all. These are minimal; B alone lacks A/C, A alone lacks B.

$B \rightarrow D$: B is not a superkey and D is nonprime (prime attributes are A,B,C because they occur in candidate keys AB/BC). Therefore it violates both 3NF and BCNF.

A clean lossless separation is $R_1(A, B, C)$, $R_2(B, D)$, $R_3(D, E)$, $R_4(E, F)$. Each step can be obtained by decomposing on an FD whose determinant is the intersection, so the chain of decompositions is lossless; dependencies are also directly represented.

Question 7 - Concurrency Control Execution**[1 pt]**

Solution. Both transactions read $\text{Seats}=1$ before either writes. Each decides a seat is available and computes 0. T_1 writes 0; T_2 then writes 0 (or reverse). Final stored $\text{Seats}=0$, but two seats were sold from one available seat. This is a lost-update/check-then-act race. Serializable isolation, an atomic conditional update such as `UPDATE ... SET Seats=Seats-1 WHERE Seats>0`, or locking the row before the check prevents it.

Question 8 - Query Optimization**[1 pt]**

Solution. Uniform ReviewerID selectivity gives $3,000,000/30,000 = 100$ rows per reviewer, roughly up to 100 data-page fetches if unclustered. B+ tree cost about $3 + \text{up to } 100$; hash is typically a little cheaper for equality (bucket lookup + those data pages). Full scan is 30,000 pages, so either equality index wins strongly; hash is usually preferred for pure equality.

For $\text{Score}=5$, about $20\% = 600,000$ rows, i.e. 6,000 pages if perfectly clustered but potentially a very large fraction of the 30,000 pages with an unclustered index. Random RID lookups can cost more than

a sequential full scan, so the optimizer may scan. Low cardinality and poor clustering make the index unattractive.

– *End of Solutions* –